Postgres

PostgreSQL Cheatsheet

Connection & Basic Commands

Connecting to PostgreSQL

```
# Connect via psql
psql -h localhost -p 5432 -U username -d database_name

# Connect with password prompt
psql -h localhost -p 5432 -U username -d database_name -W

# Connect via Docker container
docker exec -it container_name psql -U username -d database_name

# Connect with connection string
psql "postgresql://username:password@localhost:5432/database_name"
```

Basic psql Commands

```
-- Exit psql
\q

-- List all databases
\l

-- Connect to a database
\c database_name

-- List all tables
\dt

-- List all schemas
```

```
\dn

-- Describe table structure
\d table_name

-- Show table with indexes and constraints
\d+ table_name

-- List all users/roles
\du

-- Show current user
SELECT current_user;

-- Show current database
SELECT current_database();

-- Clear screen
\! clear

-- Help
\?

-- Execute SQL from file
\i /path/to/file.sql

-- Time queries
\timing on
```

Database Operations

Create & Drop Database

```
-- Create database
CREATE DATABASE my_database;
CREATE DATABASE my_database WITH ENCODING 'UTF8';
```

```
-- Drop database
DROP DATABASE my_database;

-- Create database with owner
CREATE DATABASE my_database OWNER my_user;
```

Users & Permissions

```
-- Create user
CREATE USER my_user WITH PASSWORD 'my_password';

-- Create user with privileges
CREATE USER my_user WITH PASSWORD 'my_password' CREATEDB CREATEROL
E;

-- Grant privileges
GRANT ALL PRIVILEGES ON DATABASE my_database TO my_user;
GRANT SELECT, INSERT, UPDATE ON table_name TO my_user;
GRANT ALL ON ALL TABLES IN SCHEMA public TO my_user;

-- Remove privileges
REVOKE ALL PRIVILEGES ON DATABASE my_database FROM my_user;

-- Drop user
DROP USER my_user;

-- Change password
ALTER USER my_user PASSWORD 'new_password';
```

Table Operations

Create Table

```
-- Basic table
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
```

```
    name VARCHAR(100) NOT NULL,
    email VARCHAR(255) UNIQUE,
    age INTEGER,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Table with constraints
CREATE TABLE orders (
    id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,
    total DECIMAL(10,2) NOT NULL CHECK (total > 0),
    status VARCHAR(20) DEFAULT 'pending'
);

-- Create table from query
CREATE TABLE new_table AS SELECT * FROM existing_table WHERE condition;
```

Alter Table

```
-- Add column
ALTER TABLE users ADD COLUMN phone VARCHAR(20);

-- Drop column
ALTER TABLE users DROP COLUMN phone;

-- Rename column
ALTER TABLE users RENAME COLUMN name TO full_name;

-- Change column type
ALTER TABLE users ALTER COLUMN age TYPE BIGINT;

-- Add constraint
ALTER TABLE users ADD CONSTRAINT unique_email UNIQUE (email);

-- Drop constraint
ALTER TABLE users DROP CONSTRAINT unique_email;
```

```
-- Rename table
ALTER TABLE users RENAME TO customers;
```

Drop Table

```
-- Drop table
DROP TABLE table_name;

-- Drop table if exists
DROP TABLE IF EXISTS table_name;

-- Drop table cascade (removes dependent objects)
DROP TABLE table_name CASCADE;
```

Data Operations

Insert Data

```
-- Single insert
INSERT INTO users (name, email, age) VALUES ('John Doe', 'john@email.com', 3
0);

-- Multiple inserts
INSERT INTO users (name, email, age) VALUES
    ('Jane Smith', 'jane@email.com', 25),
    ('Bob Johnson', 'bob@email.com', 35);

-- Insert from select
INSERT INTO new_users SELECT * FROM users WHERE age > 25;

-- Insert and return data
INSERT INTO users (name, email) VALUES ('Alice', 'alice@email.com') RETURNING
id, created_at;
```

Update Data

```
-- Basic update
UPDATE users SET age = 31 WHERE name = 'John Doe';

-- Update multiple columns
UPDATE users SET age = 32, email = 'newemail@example.com' WHERE id = 1;

-- Update with calculation
UPDATE products SET price = price * 1.1;

-- Update with subquery
UPDATE users SET age = (SELECT AVG(age) FROM users) WHERE age IS NULL;

-- Update and return
UPDATE users SET age = 35 WHERE id = 1 RETURNING *;
```

Delete Data

```
-- Delete specific rows
DELETE FROM users WHERE age < 18;

-- Delete all rows (but keep table structure)
DELETE FROM users;

-- Delete with subquery
DELETE FROM orders WHERE user_id IN (SELECT id FROM users WHERE age < 1
8);

-- Delete and return
DELETE FROM users WHERE id = 1 RETURNING *;
```

Select Data

```
-- Basic select
SELECT * FROM users;
SELECT name, email FROM users;
```

```
-- With conditions
SELECT * FROM users WHERE age > 25;
SELECT * FROM users WHERE name LIKE 'John%';
SELECT * FROM users WHERE age BETWEEN 25 AND 35;
SELECT * FROM users WHERE email IS NOT NULL;

-- Ordering
SELECT * FROM users ORDER BY age DESC;
SELECT * FROM users ORDER BY name ASC, age DESC;

-- Limiting
SELECT * FROM users LIMIT 10;
SELECT * FROM users OFFSET 5 LIMIT 10;

-- Grouping
SELECT age, COUNT(*) FROM users GROUP BY age;
SELECT age, COUNT(*) FROM users GROUP BY age HAVING COUNT(*) > 1;

-- Joins
SELECT u.name, o.total
FROM users u
JOIN orders o ON u.id = o.user_id;

SELECT u.name, o.total
FROM users u
LEFT JOIN orders o ON u.id = o.user_id;
```

Advanced Queries

Window Functions

```
-- Row numbers
SELECT name, age, ROW_NUMBER() OVER (ORDER BY age) as rank FROM users;

-- Running totals
SELECT date, amount, SUM(amount) OVER (ORDER BY date) as running_total FRO
M sales;
```

```
-- Ranking
SELECT name, salary, RANK() OVER (ORDER BY salary DESC) as rank FROM empl
oyees;
```

CTEs (Common Table Expressions)

```
-- Basic CTE
WITH young_users AS (
    SELECT * FROM users WHERE age < 30
)
SELECT * FROM young_users WHERE email LIKE '%gmail.com';

-- Recursive CTE
WITH RECURSIVE series AS (
    SELECT 1 as num
    UNION ALL
    SELECT num + 1 FROM series WHERE num < 10
)
SELECT * FROM series;
```

Subqueries

```
-- Subquery in WHERE
SELECT * FROM users WHERE id IN (SELECT user_id FROM orders WHERE total >
100);

-- Correlated subquery
SELECT * FROM users u WHERE EXISTS (SELECT 1 FROM orders o WHERE o.user
_id = u.id);

-- Subquery in SELECT
SELECT name, (SELECT COUNT(*) FROM orders WHERE user_id = users.id) as or
der_count FROM users;
```

Data Types

Common Data Types

```
-- Numeric
SMALLINT, INTEGER, BIGINT
DECIMAL(precision, scale), NUMERIC(precision, scale)
REAL, DOUBLE PRECISION
SERIAL, BIGSERIAL

-- Character
CHAR(n), VARCHAR(n), TEXT

-- Date/Time
DATE, TIME, TIMESTAMP, TIMESTAMPTZ, INTERVAL

-- Boolean
BOOLEAN (TRUE/FALSE/NULL)

-- JSON
JSON, JSONB

-- Arrays
INTEGER[], TEXT[], etc.

-- UUID
UUID
```

Working with JSON

```
-- Create table with JSON
CREATE TABLE products (
    id SERIAL PRIMARY KEY,
    data JSONB
);

-- Insert JSON
INSERT INTO products (data) VALUES ('{"name": "iPhone", "price": 999, "tags":
["phone", "apple"]}');
```

```
-- Query JSON
SELECT data->>'name' as product_name FROM products;
SELECT * FROM products WHERE data->'price' > '500';
SELECT * FROM products WHERE data ? 'name';
SELECT * FROM products WHERE data @> '{"tags": ["phone"]}';
```

Indexes

Create Indexes

```
-- Basic index
CREATE INDEX idx_users_email ON users(email);

-- Unique index
CREATE UNIQUE INDEX idx_users_email_unique ON users(email);

-- Composite index
CREATE INDEX idx_users_name_age ON users(name, age);

-- Partial index
CREATE INDEX idx_active_users ON users(email) WHERE active = true;

-- Expression index
CREATE INDEX idx_users_lower_email ON users(LOWER(email));

-- JSON index
CREATE INDEX idx_products_name ON products USING GIN ((data->>'name'));
```

Manage Indexes

```
-- List indexes
\di

-- Drop index
DROP INDEX idx_users_email;
```

```
-- Reindex
REINDEX INDEX idx_users_email;
REINDEX TABLE users;
```

Functions & Procedures

String Functions

```
-- Concatenation
SELECT CONCAT(first_name, ' ', last_name) FROM users;
SELECT first_name || ' ' || last_name FROM users;

-- String manipulation
SELECT UPPER(name), LOWER(email) FROM users;
SELECT LENGTH(name), SUBSTRING(name, 1, 3) FROM users;
SELECT TRIM(name), REPLACE(email, '@gmail.com', '@example.com') FROM user
s;
```

Date Functions

```
-- Current date/time
SELECT NOW(), CURRENT_DATE, CURRENT_TIME;

-- Date arithmetic
SELECT NOW() + INTERVAL '1 day';
SELECT NOW() - INTERVAL '1 month';

-- Extract parts
SELECT EXTRACT(YEAR FROM created_at) FROM users;
SELECT DATE_PART('month', created_at) FROM users;

-- Formatting
SELECT TO_CHAR(created_at, 'YYYY-MM-DD') FROM users;
```

Aggregate Functions

```
SELECT COUNT(*), COUNT(DISTINCT email) FROM users;
SELECT SUM(total), AVG(total), MIN(total), MAX(total) FROM orders;
SELECT STRING_AGG(name, ', ') FROM users;
SELECT ARRAY_AGG(name) FROM users;
```

Performance & Maintenance

Query Analysis

```
-- Explain query plan
EXPLAIN SELECT * FROM users WHERE age > 25;
EXPLAIN ANALYZE SELECT * FROM users WHERE age > 25;

-- Query statistics
SELECT * FROM pg_stat_user_tables;
SELECT * FROM pg_stat_user_indexes;
```

Maintenance

```
-- Update table statistics
ANALYZE users;
ANALYZE; -- all tables

-- Vacuum (cleanup)
VACUUM users;
VACUUM FULL users; -- reclaims space but locks table

-- Auto vacuum settings
SELECT * FROM pg_settings WHERE name LIKE '%autovacuum%';
```

Database Information

```
-- Database size
SELECT pg_size_pretty(pg_database_size('database_name'));
```

```
-- Table sizes
SELECT
    schemaname,
    tablename,
    pg_size_pretty(pg_total_relation_size(schemaname||'.'||tablename)) as size
FROM pg_tables
ORDER BY pg_total_relation_size(schemaname||'.'||tablename) DESC;

-- Connection info
SELECT * FROM pg_stat_activity;
```

Backup & Restore

Using pg_dump

```
# Backup database
pg_dump -h localhost -U username database_name > backup.sql
pg_dump -h localhost -U username -Fc database_name > backup.dump

# Backup specific table
pg_dump -h localhost -U username -t table_name database_name > table_backu
p.sql

# Restore
psql -h localhost -U username database_name < backup.sql
pg_restore -h localhost -U username -d database_name backup.dump
```

Copy Data

```
-- Export to CSV
COPY users TO '/path/to/users.csv' WITH CSV HEADER;

-- Import from CSV
COPY users FROM '/path/to/users.csv' WITH CSV HEADER;

-- Copy between tables
```

```
COPY (SELECT * FROM users WHERE age > 25) TO '/path/to/filtered_users.csv' W
ITH CSV HEADER;
```

Common Patterns

Upsert (INSERT ... ON CONFLICT)

```
INSERT INTO users (id, name, email)
VALUES (1, 'John Doe', 'john@example.com')
ON CONFLICT (id)
DO UPDATE SET
   name = EXCLUDED.name,
   email = EXCLUDED.email;
```

Pagination

```
-- Offset-based (can be slow for large offsets)
SELECT * FROM users ORDER BY id LIMIT 20 OFFSET 100;

-- Cursor-based (faster for large datasets)
SELECT * FROM users WHERE id > 100 ORDER BY id LIMIT 20;
```

Conditional Logic

```
-- CASE statements
SELECT
   name,
   CASE
      WHEN age < 18 THEN 'Minor'
      WHEN age < 65 THEN 'Adult'
      ELSE 'Senior'
   END as age_group
FROM users;

-- COALESCE (first non-null value)
```

```
SELECT COALESCE(nickname, first_name, 'Unknown') as display_name FROM use
rs;
```

Environment Variables for Connection

```
export PGHOST=localhost
export PGPORT=5432
export PGDATABASE=mydb
export PGUSER=myuser
export PGPASSWORD=mypassword

# Now you can just run:
psql
```

Quick Docker PostgreSQL Setup

```
# Run PostgreSQL in Docker
docker run --name postgres-lab \
  -e POSTGRES_DB=labdb \
  -e POSTGRES_USER=labuser \
  -e POSTGRES_PASSWORD=password123 \
  -p 5432:5432 \
  -d postgres:15

# Connect to it
docker exec -it postgres-lab psql -U labuser -d labdb
```